

Semana 2 — CharacterBody3D, movimentação e Input Map

Semana 2

CharacterBody3D, movimentação e Input Map

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade I — Aprender a Ferramenta (Semanas 1–3)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender por que uma engine desacopla a intenção do jogador da ação no mundo
- Configurar um CharacterBody3D controlável usando `move_and_slide`
- Configurar um Input Map e conectar Actions à movimentação do Player

Semana 2 — CharacterBody3D, movimentação e Input Map

ENCONTRO 1

CharacterBody3D e move_and_slide

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão da Scene da Semana 1 (10 min)
- Introdução: por que engines desacoplam intenção de ação (15 min)
- Demonstração: CharacterBody3D + CollisionShape3D via Orchestrator (35 min)
- Laboratório: cada estudante monta seu Player (45 min)
- Desafio: ajustar forma de colisão (20 min)
- Feedback e fechamento (10 min)

Semana 2 — CharacterBody3D, movimentação e Input Map



Como um personagem sabe que não deve atravessar uma parede?

Pense em qualquer jogo que vocês já jogaram, de qualquer engine.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema Universal da Locomoção

Todo personagem controlável — jogador ou inimigo — precisa resolver o mesmo problema físico:

- Mover-se no mundo sem atravessar paredes
- Deslizar suavemente ao encostar em obstáculos
- Responder corretamente a rampas e desníveis

Reimplementar isso do zero a cada projeto seria caro e repetitivo.

CharacterBody3D e move_and_slide

- **CharacterBody3D** — Node especializado em corpos controlados por código
- Diferente do RigidBody3D, que é simulado livremente pela física
- `move_and_slide()` — resolve deslocamento e colisão em uma única chamada, a cada frame de física
- Ao final deste encontro, o Player existe e é sólido — mas ainda não se move sozinho

Locomoção no Godot × Unity

Godot

- CharacterBody3D já pronto
- `move_and_slide()` resolve tudo em uma chamada
- Solução de locomoção embutida no próprio Node

Unity

- CharacterController **ou** Rigidbody + script próprio
- Sem um único componente "pronto" equivalente
- Mais decisões de arquitetura recaem sobre o time

Demonstração — Montagem do Player

O que será construído:

- Node `Player` (`CharacterBody3D`), filho de `NívelTeste`
- `CollisionShape3D` — forma física de colisão
- `Malha` (`MeshInstance3D`) — representação visual
- Orchestration `player.torch` chamando `move_and_slide` no `PhysicsProcess`
- `Player` salvo como Scene própria (`scenes/characters/Player.tscn`), reaproveitável em outros níveis

Por quê: primeiro personagem controlável do semestre, base direta do Input Map no Encontro 2.

Imagem sugerida

*Objetivo: mostrar a árvore de cena com **Player** (CharacterBody3D) já contendo **CollisionShape3D** e **Malha**, ao lado do Viewport 3D com a cápsula posicionada sobre o **Chao**.*

Enquadramento: captura de tela dividida — Scene dock à esquerda, Viewport 3D à direita.

*Elementos presentes: hierarquia **Player** > **CollisionShape3D**, **Malha**, cápsula alinhada ao chão no Viewport.*

Destaque visual: contorno colorido separando forma de colisão (vermelho, semi-transparente) da malha visual (azul).

Legenda sugerida: "Player montado: colisão e malha visual como Nodes separados, alinhados sobre o chão."

Laboratório — Montagem do Player

Cada estudante replica, no próprio projeto:

1. `Player` (`CharacterBody3D`) como filho de `NivelTeste`
2. `CollisionShape3D` com uma Shape atribuída (ex.: `CapsuleShape3D`)
3. `Malha` (`MeshInstance3D`) com uma Mesh atribuída, alinhada à colisão
4. Orchestration `player.torch` chamando `move_and_slide` no `PhysicsProcess`
5. `Player` salvo como Scene própria em `scenes/characters/Player.tscn`

Boas Práticas — Colisão e Composição

- Separar sempre `CollisionShape3D` (física) de `MeshInstance3D` (visual), mesmo quando parecem redundantes
- Nomear o Node raiz como `Player`, nunca deixar o nome padrão `CharacterBody3D`
- Confirmar visualmente o alinhamento entre colisão e malha antes de avançar
- Associar a Orchestration ao Node correto (`Player`, não `NívelTeste`)



Desafio — Encontro 1

Ajuste a forma ou o tamanho da `CollisionShape3D` do próprio Player — por exemplo, cápsula versus caixa, ou uma escala diferente da demonstrada.

OBJETIVOS

Justifique brevemente a escolha em relação ao personagem que pretende usar no Vertical Slice. Não há solução única.

Fechamento — Encontro 1

- Player (CharacterBody3D) montado, sólido, alinhado ao `Chao`
- `move_and_slide` já chamado a cada frame de física, ainda sem velocity
- Próximo passo: conectar o Input Map à movimentação, no Encontro 2

Semana 2 — CharacterBody3D, movimentação e Input Map

ENCONTRO 2

Input Map e InputEvent

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (Player parado) (10 min)
- Introdução: por que desacoplar dispositivo físico de ação lógica (20 min)
- Demonstração: Input Map + conexão com move_and_slide (35 min)
- Laboratório: cada estudante configura o próprio Input Map (45 min)
- Desafio: Action adicional (correr, agachar ou pular) (20 min)
- Feedback e fechamento (5 min)

Semana 2 — CharacterBody3D, movimentação e Input Map



Se o código do jogo checasse diretamente a tecla W, o que aconteceria ao tentar suportar um gamepad?

Pense antes de abrir o Project Settings.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Input Map — Camada de Desacoplamento

Se o código lesse diretamente "tecla W pressionada", qualquer remapeamento exigiria reescrever a lógica do jogo inteira.

- O jogador aperta uma tecla física
- A engine traduz isso em uma **Action** nomeada (ex.: "mover para frente")
- O código de gameplay consome a Action — nunca a tecla em si

Input Map e InputEvent no Godot

- **Input Map** — configurado uma vez em Project Settings, associa teclas a Actions
- **InputEvent** — evento gerado a cada interação do jogador, traduzido automaticamente para a Action
- A Orchestration só pergunta: "a Action `move_forward` está pressionada agora?"
- Actions desta semana: `move_forward`, `move_back`, `move_left`, `move_right`

Input Map no Godot × Unity

Godot

- Input Map único e global do projeto
- Actions configuradas em um só lugar
- Mais simples para o caso de um único jogador

Unity

- Input System com Action Maps
- Componente Player Input conecta Actions ao código
- Mais granularidade para múltiplos esquemas/dispositivos

Diagrama Sugerido — Fluxo do Input

Diagrama sugerido

Fluxo linear: `Tecla física (W)` → `InputEvent` → `Action (move_forward)` → `Orchestration`
`lê a Action` → `velocity atribuída` → `move_and_slide()`.

Objetivo: visualizar as camadas entre o dispositivo físico e o movimento efetivo do Player.

Legenda sugerida: "Da tecla física ao movimento: cada seta é uma camada de desacoplamento."

Demonstração — Input Map e Conexão

O que será construído:

- Quatro Actions no Input Map: `move_forward`, `move_back`, `move_left`, `move_right`
- Leitura das Actions dentro da Orchestration `player.torch`
- Combinação em um vetor de direção, aplicado à `velocity` do Player
- `move_and_slide` chamado após a `velocity` ser atribuída

Por quê: transforma o Player sólido do Encontro 1 em um Player efetivamente controlável.

Imagem sugerida

Objetivo: mostrar a aba Input Map do Project Settings, com as quatro Actions de movimentação já configuradas.

Enquadramento: captura de tela da janela Project Settings, aba Input Map.

Elementos presentes: lista de Actions (`move_forward`, `move_back`, `move_left`, `move_right`), teclas associadas a cada uma.

Destaque visual: o botão "Add" usado para criar uma nova Action.

Legenda sugerida: "Input Map do projeto com as Actions de movimentação configuradas."

Laboratório — Input Map e Movimentação

Cada estudante configura, no próprio projeto:

1. Quatro Actions no Input Map (`move_forward` , `move_back` , `move_left` , `move_right`)
2. Leitura das Actions na Orchestration `player.torch`
3. Vetor de direção combinado e aplicado à `velocity`
4. Teste com F6, movendo o Player nas quatro direções

Boas Práticas — Nomenclatura de Input

- Nomear Actions por intenção do jogador (`move_forward`), nunca pela tecla física
- Centralizar toda leitura de input do Player dentro da própria Orchestration
- Testar cada Action isoladamente antes de combinar as quatro em um vetor
- Evitar espalhar chamadas de Input Map por múltiplos Nodes



Desafio — Encontro 2

Adicione uma nova Action ao Input Map, não demonstrada em aula — correr, agachar ou pular.

OBJETIVOS

Liberdade de implementação: correr como multiplicador de velocidade, pular como impulso vertical simples. Não há solução única.

Resultado Esperado da Semana

- Player (CharacterBody3D) montado, com `CollisionShape3D` e `Malha` alinhados
- Input Map com quatro Actions de movimentação, mais uma Action do desafio
- Orchestration `pPlayer.torch` lendo o Input Map e movendo o Player via `move_and_slide`
- Turma relaciona CharacterBody3D/Input Map aos equivalentes na Unity

Checklist da Semana

- [] `Player` (`CharacterBody3D`) com `CollisionShape3D` e `Malha` alinhados, salvo em `scenes/characters/Player.tscn`
- [] Orchestration `player.torch` associada ao `Player`
- [] Input Map com `move_forward`, `move_back`, `move_left`, `move_right`
- [] `velocity` atribuída antes de `move_and_slide`
- [] `Player` se move nas quatro direções, sem erros
- [] Action adicional do desafio (correr, agachar ou pular)

Semana 2 — CharacterBody3D, movimentação e Input Map

Próximos Passos — Semana 3

O Player controlável desta semana é a base direta da Semana 3:

- Materiais e terreno (**Terrain3D**)
- Iluminação global (**SDFGI/VoxelGI**)
- Exportação do primeiro build executável, encerrando o Módulo 1

Leitura recomendada: Godot Docs — Physics (CharacterBody3D) e Inputs; Unity Manual (consulta comparativa) — Input System.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos